- **My phase estimates of 8-10 weeks each are conservative-optimistic, not pessimistic.**** **Engineering projects routinely overrun. If you need to commit dates to anyone external, double these estimates and treat the original as stretch.

- **No formal**** ****‘****v6.0 launch event.****’**** **The platform goes live with Phase 1 (Prong 1 only) and grows. There’s no big-bang launch — there’s ‘first booking’ then ‘first outside tenant onboarded’ then ‘tenth tenant active’ then ‘first month of profitable operation.’

- **Phase 3 length is the most uncertain.**** **Heavy production data informs scope. Could be 6 weeks if things go smoothly or 16 weeks if Anthropic pricing changes or scale exposes issues.

Section 32 — Self-Service Help

The Self-Service Help feature reduces support load on tenant admins and captures customer-reported defects with enough structure that engineering can act on them automatically where possible. It serves three outcomes: educate users, capture defects, and capture feature requests. Each outcome has a dedicated flow tuned for that purpose.

This section also defines how tenant-facing documentation is generated, indexed in the RAG service under a platform-docs scope, and kept current as code changes.

32.1 Purpose and Scope

32.1.1 Purpose

The Self-Service Help feature exists to reduce the operational load of supporting tenant admins and to capture customer-reported defects with enough structure that engineering can act on them automatically where possible. It serves three distinct outcomes: educate users, capture defects, and capture feature requests. Each gets a dedicated flow tuned for that outcome.

32.1.2 Scope

In scope for this section:

- A new Help section inside the tenant admin console with onscreen, printable, and downloadable documentation.

- Three AI-driven entry points for tenant admins: ‘I need help,’ ‘Report a bug,’ and ‘Request a feature.’

- A parallel customer bug-reporting path that runs through the existing travel concierge AI persona.

- GitHub integration for bug and feature-request issue creation.

- A bug-auto-fix evaluation pipeline that integrates with the existing CI/CD release pipeline.

- Abuse monitoring extensions for help-flow submissions.

Out of scope for this section (deferred):

- Authoring the actual help documentation content (engineering builds the structure; content is a separate workstream).
- Non-English documentation. Per v6 §25.8, the platform is English-only at launch.
- Integration with bug trackers other than GitHub.
- Customer-facing feature-request voting or roadmap visibility.
- Email notifications for issue lifecycle events beyond standard GitHub notification flow.

32.1.3 Personas Affected

This feature touches the personas defined in v6 §1.3 as follows:

Persona	What changes
Customer	Gains a bug-reporting path via the travel concierge AI persona. Authentication required (OAuth, per §17 of v6 spec).
Tenant Owner	Gains a /admin/help route with documentation and three help flows.
Tenant Agent	Same as Tenant Owner; can use help flows but cannot change tenant-wide settings.
Tenant Billing Admin	Same as Tenant Owner for help purposes; billing flow unchanged.
Platform Super Admin	Gains a triage view of all bug/feature submissions across tenants.
Platform Support	Gains visibility into open help sessions for assisted resolution.
AI Persona (travel concierge)	Gets a new bug-report intent recognizer. Existing personas unchanged otherwise.
Help AI (new)	New persona type, scoped only to help/bug/feature flows. Runs under the supervisor (v6 §10).

32.1.4 Cross-References

This feature explicitly depends on, or interacts with:

- **v6 §1.5** — multi-tenancy and RLS patterns. All new tables follow these.
- **v6 §6** — RAG service. The help documentation may be indexed for retrieval by the Help AI.
- **v6 §10** — Supervisor. All Help AI chats run through the supervisor.

- **v6 §11** — Customer memory. Bug submissions from customers are linked to the customer's memory only as audit, not as conversational facts.
- **v6 §17** — Authentication. Customer bug reports require OAuth authentication.
- **v6 §22** — Ingestion pipeline / PII redaction. Bug-report bodies pass through this before issue creation.
- **v6 §26** — Roles and permissions.
- **v6 §27** — Abuse monitoring. A new dimension is added.
- **CI/CD §3, §4, §5** — GitHub Environments, Actions workflow, staging DB copy. The auto-fix pipeline integrates here.

32.2 User Experience Overview

32.2.1 Entry Points

Three entry points for tenant admins (all under /admin/help):

- Documentation viewer — the default view. Renders the help docs onscreen.
- 'I need help' button — opens the Help AI in conversational mode.
- 'Report a bug' button — opens the Help AI in structured bug-gathering mode.
- 'Request a feature' button — opens the Help AI in structured feature-gathering mode.

One entry point for customers (via existing chat surface):

- Customer mentions a defect ('this looks broken,' 'something's wrong,' 'I think there's a bug') — the travel concierge persona offers to file a bug. If the customer accepts, the Help AI bug flow takes over within the same chat surface.

32.2.2 The Documentation Viewer

The viewer renders a set of Markdown documents organized into sections. Each section corresponds to a major feature area of the tenant admin console (e.g., 'Setting up branding,' 'Managing your CRM,' 'Understanding your usage and tier'). Source files live in apps/main/content/help/ in the platform repo and are rendered with a standard Markdown-to-HTML pipeline (e.g., remark/rehype).

The viewer offers three output modes:

- **Onscreen** — default. Navigation in left sidebar, content in right pane. Search across all docs.
- **Print** — applies a print-friendly stylesheet (single-column, no nav, page breaks at section boundaries). User invokes via browser print.

- **Download as PDF** — server-side render of the entire doc set as a single PDF, generated on demand and cached for 1 hour. Filename: {tenant_slug}-admin-guide-{date}.pdf.
- **Download as Word** — server-side render of the entire doc set as a single .docx file. Same filename pattern, .docx extension.

32.2.3 Help AI Chat Surface

When a help/bug/feature button is clicked, a slide-over chat panel appears (right side of viewport, ~480px wide on desktop, full-screen on mobile). The chat is conducted by the Help AI persona (distinct from the travel concierge personas). The panel includes the current conversation, an input field, an 'Escalate to human' button, and a 'Close' button.

All Help AI chats run through the supervisor (v6 §10) the same way customer-facing chats do. Same hallucination checks, same kill switch, same audit trail.

32.3 Tenant Admin Console Documentation

32.3.1 Documentation Source of Truth

All help documentation lives as Markdown files in apps/main/content/help/ in the platform monorepo. This co-locates documentation with the code that it documents, so feature changes can update docs in the same pull request. Documentation is part of the release pipeline — it ships with the code.

Documentation files are NOT separately versioned; they version with the platform code. The platform displays the docs that correspond to the currently-deployed code version.

32.3.2 Document Structure

Initial doc set covers (file names are illustrative; subject to refinement during implementation):

File	Topic
01-getting-started.md	First-time setup, signup completion, initial configuration
02-tenant-settings.md	Tenant settings: business identity, contact info, time zone
03-branding.md	White-label branding: logos, colors, custom domain setup
04-personas.md	Configuring AI personas for your tenant
05-crm.md	Using the CRM: contacts, relationships, pipeline
06-quotes-and-bookings.md	Quote builder, booking submission, tracking
07-rag-content.md	Submitting content to your RAG knowledge base; review queue

08-usage-and-billing.md	Understanding your tier, usage metrics, billing, and subscription management (changing tier, adding/removing seats, switching billing period)
09-team-and-permissions.md	Adding team members, role assignments
10-customer-management.md	Customer accounts, memory, escalations
11-supervisor-and-quality.md	Reviewing supervisor findings, handling escalations
12-troubleshooting.md	Common issues and how to diagnose them

32.3.3 Print and Download Generation

Print: applies a CSS @media print stylesheet. No server work needed for print; the browser handles it.

PDF generation:

- Concatenates all Markdown files in section order into a single document.
- Renders to HTML via remark/rehype.
- Converts HTML to PDF via Puppeteer (headless Chromium) on a serverless function.
- Caches the result in Supabase Storage with 1-hour TTL keyed by code version + tenant_id.
- Returns a signed URL for download.

.docx generation:

- Concatenates all Markdown files in section order.
- Converts to .docx via docx-js with platform-branded styles.
- Caches the result the same way as PDF.
- Returns a signed URL for download.

Both downloads include the tenant's branded header (logo, business name) if branding is configured. Otherwise default platform branding.

32.3.4 Help Documentation as RAG Content

The help documentation IS indexed in the RAG service (v6 §6), scoped to a special 'platform-docs' scope visible to all tenants. The Help AI uses this RAG corpus when responding to 'I need help' queries. This is the only scope outside of 'global' and 'tenant' (v6 §6.9 establishes the strictly-two-level scope; the platform-docs scope is documented here as a third logical scope used only by the Help AI).

Note: ** **this is the one place where Section 32 extends v6 §6's two-scope model. The platform-docs scope is read-only and managed entirely by the platform release pipeline. It is not visible to tenant admins as content they can submit or review.

32.4 Help AI Persona

32.4.1 Role and Boundaries

The Help AI is a dedicated AI persona distinct from the customer-facing travel concierge personas. It has access to the platform-docs RAG scope and to a structured information-gathering protocol for bug and feature flows. It does NOT have access to tenant customer data, CRM contents, or commission data — its scope is the platform itself, not the tenant's business.

32.4.2 System Prompt Outline

The Help AI runs with a system prompt that includes:

- Role: 'You are a help assistant for the AI Travel Concierge platform.'
- Capabilities: search platform docs, gather structured info for bugs/features, escalate to humans.
- Boundaries: do not invent feature behaviors; cite docs where possible; if uncertain, say so.
- Tone: professional, brief, helpful. No marketing language.
- PII handling: redact any PII the user enters before storing or sending to GitHub.

32.4.3 The Three Flows

Help flow

Open conversational Q&A. The Help AI searches the platform-docs RAG, returns answers with doc references, suggests follow-up topics. If it cannot answer with sufficient confidence after 3 messages, it offers to escalate to platform support.

Bug flow

Structured information gathering. The Help AI asks for the following, one at a time, persisting answers as it goes:

- Where in the platform is the problem? (URL or feature name)
- What is it doing? (the actual behavior)
- What should it be doing? (the expected behavior)
- Steps to reproduce, as specifically as possible.
- How often does this happen? (always / sometimes / once)
- Browser/device. (auto-detect; offered for user to confirm)
- Optional: screenshots.

After gathering, the Help AI computes a confidence/clarity score (see §9.2) and shows the user a summary before submission. The user can edit before submitting. On submit, the bug is created as a GitHub

issue per §7.

Feature flow

Structured information gathering, lighter weight than bug flow:

- What do you want to do?
- Why? What problem does it solve for you?
- How do you handle this today (workaround)?
- How often would you use this?

On submit: GitHub issue with label feature-request, awaiting human review.

32.4.4 Cost Model

Help AI calls follow the same cost-attribution pattern as customer-facing AI (v6 §27.12). They attribute to the tenant whose user is interacting. The Help AI uses Sonnet for the main response and Haiku for the supervisor preflight, same as customer-facing chat.

Customer-side bug flows attribute to the tenant that owns the customer, not to a platform pool. This matters for abuse monitoring (§11) and gives tenants visibility into help-related cost.

32.5 Database Schema

32.5.1 help_sessions

One row per help-chat session. A session is opened when any help/bug/feature flow starts and closed when the user leaves the chat surface or submits.

CREATE TABLE public.help_sessions (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), tenant_id UUID NOT NULL REFERENCES public.tenants(id), user_id UUID NOT NULL REFERENCES public.users(id), session_type TEXT NOT NULL CHECK (session_type IN (‘help’,‘bug’,‘feature’)), source_surface TEXT NOT NULL CHECK (source_surface IN (‘admin’,‘customer_chat’)), conversation_id UUID REFERENCES public.conversations(id), started_at TIMESTAMPTZ NOT NULL DEFAULT NOW(), ended_at TIMESTAMPTZ, outcome TEXT CHECK (outcome IN (‘resolved’,‘submitted’,‘escalated’,‘abandoned’)), ai_messages_count INTEGER DEFAULT 0, escalated_to_human BOOLEAN DEFAULT FALSE, escalation_reason TEXT);

CREATE INDEX help_sessions_tenant_idx ON public.help_sessions(tenant_id); CREATE INDEX help_sessions_user_idx ON public.help_sessions(user_id); CREATE INDEX help_sessions_type_idx ON public.help_sessions(tenant_id, session_type, started_at DESC);

32.5.2 bug_submissions

CREATE TABLE public.bug_submissions (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), help_session_id UUID NOT NULL REFERENCES public.help_sessions(id), tenant_id UUID NOT NULL REFERENCES public.tenants(id), submitter_user_id UUID NOT NULL REFERENCES public.users(id), source_type TEXT NOT NULL CHECK (source_type IN (‘tenant_admin’,‘tenant_agent’,‘customer’)),

– Structured fields (gathered by Help AI; redacted of PII) where_in_platform TEXT, actual_behavior TEXT, expected_behavior TEXT, steps_to_reproduce TEXT, frequency TEXT CHECK (frequency IN (‘always’,‘sometimes’,‘once’,‘unknown’)), browser_info JSONB, screenshots JSONB, – array of storage URLs

– Quality scoring confidence_score NUMERIC(3,2) CHECK (confidence_score >= 0 AND confidence_score <= 1), confidence_factors JSONB,

– GitHub linkage github_issue_number INTEGER, github_issue_url TEXT, github_issue_state TEXT CHECK (github_issue_state IN (‘pending’,‘open’,‘closed’,‘failed’)), github_creation_error TEXT,

– Lifecycle submitted_at TIMESTAMPTZ DEFAULT NOW(), closed_at TIMESTAMPTZ, resolution_summary TEXT);

CREATE INDEX bug_submissions_tenant_idx ON public.bug_submissions(tenant_id); CREATE INDEX bug_submissions_state_idx ON public.bug_submissions(github_issue_state, submitted_at DESC);

32.5.3 feature_requests

CREATE TABLE public.feature_requests (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), help_session_id UUID NOT NULL REFERENCES public.help_sessions(id), tenant_id UUID NOT NULL REFERENCES public.tenants(id), submitter_user_id UUID NOT NULL REFERENCES public.users(id), source_type TEXT NOT NULL CHECK (source_type IN (‘tenant_admin’,‘tenant_agent’,‘customer’)),

what TEXT, why TEXT, current_workaround TEXT, expected_usage_frequency TEXT,

github_issue_number INTEGER, github_issue_url TEXT, github_issue_state TEXT CHECK (github_issue_state IN (‘pending’,‘open’,‘closed’,‘failed’)),

submitted_at TIMESTAMPTZ DEFAULT NOW(), decision TEXT CHECK (decision IN (‘accepted’,‘rejected’,‘deferred’,‘duplicate’)), decision_notes TEXT, decided_at TIMESTAMPTZ);

CREATE INDEX feature_requests_tenant_idx ON public.feature_requests(tenant_id); CREATE INDEX feature_requests_decision_idx ON public.feature_requests(decision, submitted_at DESC);

32.5.4 help_doc_versions

Caches generated PDF/.docx outputs keyed by code version and (optionally) tenant for branding.

CREATE TABLE public.help_doc_versions (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), code_version TEXT NOT NULL, tenant_id UUID REFERENCES public.tenants(id), – nullable format

TEXT NOT NULL CHECK (format IN ('pdf','docx')), storage_path TEXT NOT NULL, size_bytes BIGINT, generated_at TIMESTAMPTZ DEFAULT NOW(), expires_at TIMESTAMPTZ NOT NULL, UNIQUE (code_version, tenant_id, format));

CREATE INDEX help_doc_versions_expiry_idx ON public.help_doc_versions(expires_at);

32.5.5 Row Level Security

All four tables enable RLS. Policies follow v6 §1.5 patterns:

- Tenant scope: rows visible only to users belonging to the tenant (auth_user_in_tenant).
- Platform admin override: platform_super_admin and platform_support roles can SELECT across all tenants.
- Customer access: a customer can read their own bug_submissions and feature_requests rows but not other customers' rows in the same tenant.
- Help_doc_versions: tenant-scoped if tenant_id is set; platform-wide read if tenant_id is NULL.

32.6 API Routes

New API routes added by this feature. Follows the patterns in v6 §7.

32.6.1 Documentation Routes

Method	Path	Purpose
GET	/api/help/docs	List doc sections (titles, slugs)
GET	/api/help/docs/:slug	Get a single doc section as HTML
GET	/api/help/docs/search?q=…	Search docs; returns matching sections with snippets
POST	/api/help/docs/export	Trigger PDF or .docx generation; returns job ID
GET	/api/help/docs/export/:jobId	Poll export status; returns signed URL when ready

32.6.2 Help Chat Routes

Method	Path	Purpose
POST	/api/help/sessions	Open a new help session; returns session_id
POST	/api/help/sessions/:id/message	Send a user message; returns SSE stream of AI

		response
POST	/api/help/sessions/:id/close	Close the session with outcome
POST	/api/help/sessions/:id/escalate	Escalate to platform support

32.6.3 Bug Submission Routes

Method	Path	Purpose
POST	/api/help/bugs	Submit a bug; creates GitHub issue async
GET	/api/help/bugs/:id	Get submission state including GitHub link
GET	/api/help/bugs	List submissions for the current tenant

32.6.4 Feature Request Routes

Method	Path	Purpose
POST	/api/help/features	Submit a feature request
GET	/api/help/features/:id	Get submission state
GET	/api/help/features	List submissions for the current tenant

32.6.5 Platform Admin Routes

Method	Path	Purpose
GET	/api/admin/help/sessions	Cross-tenant view of help sessions
GET	/api/admin/help/bugs	Cross-tenant view of bug submissions
GET	/api/admin/help/features	Cross-tenant view of feature requests
PATCH	/api/admin/help/features/:id	Set decision on a feature request

32.7 GitHub Integration

32.7.1 Authentication

The platform authenticates to GitHub as a GitHub App (not a personal access token). Benefits over a PAT: per-installation scoping, finer-grained permissions, audit trail, and rotation without an individual's credentials in play.

Required GitHub App permissions:

- Repository: Issues (Read & Write) — create issues, add labels, post comments.

- Repository: Pull Requests (Read & Write) — used by the auto-fix workflow.
- Repository: Contents (Read & Write) — auto-fix branches need to push commits.
- Repository: Actions (Read) — to inspect workflow status.

32.7.2 Repository

Bug and feature-request issues are created in the same GitHub repository that holds the platform code. This is the same repo referenced in the CI/CD doc (CI/CD §4 onward). No separate repo is created for help submissions.

32.7.3 Labels

The following labels are used. The platform creates any that don't exist on first issue submission.

Label	Purpose
bug	Identifies the issue as a defect.
feature-request	Identifies the issue as a feature ask.
customer-reported	Bug or feature originated from an end customer.
tenant-admin-reported	Bug or feature originated from a tenant admin or agent.
auto-fix-candidate	Bug confidence/clarity score exceeds threshold; eligible for auto-fix.
awaiting-staging-repro	Auto-fix pipeline is attempting to reproduce on staging.
repro-confirmed	Bug reproduced on staging; draft PR opened.
repro-failed	Bug could not be reproduced; awaiting human review.
pending-human-review	Submission below auto-fix threshold; needs human triage.

32.7.4 Issue Body Format

Issues are created with a structured Markdown body. Example bug:

Source

- Type: tenant-admin-reported
- Tenant: ${tenant_slug} (id hash: ${tenant_id_hash})
- Submitter user role: tenant_owner
- Confidence/clarity score: 0.82

Where in the platform

/admin/branding

Actual behavior

The custom domain verification stays in ‘Pending’ even after I added the CNAME record yesterday.

Expected behavior

Verification should complete within a few minutes of adding the CNAME.

Steps to reproduce

- Go to /admin/branding
- Click ‘Add custom domain’
- Enter travel.example.com
- Add the CNAME at the registrar (done 2026-05-14 14:00 UTC)
- Refresh the page; status still shows ‘Pending’

Frequency

Once (it has not resolved itself).

Browser

Chrome 124 on macOS 14.5.

Submitted from help session

The auto-generated GitHub issue body MUST contain enough context that a reviewer can understand and act on the bug without clicking through to the platform admin’s internal help-session page. Include in the issue body:

- The bug description as the customer or tenant admin entered it (verbatim, redacted for PII if applicable).
- Browser, OS, viewport size from the help session metadata.
- Steps to reproduce as captured during the bug submission flow.
- Any screenshots attached (referenced inline via GitHub’s image upload mechanism, not as external links).
- Timestamp and tenant slug.

A reference to the help session is included but labeled clearly as platform-staff-only:

```
**Help session reference** (platform staff only, requires auth):
/admin/help/sessions/${session_id}
```

This link is for forensic deep-dive when needed; the issue body above should be sufficient for routine review.

Why the previous design (link-only) was insufficient

- A reviewer not on the platform admin team gets 401 on the link

with no fallback context.
- If the issue tracker is ever opened publicly (Phase 3 bug bounty), the session_id is exposed with no actionable context.
- Forensic continuity: if the help session page URL structure changes, old issues' links break.

What NOT to include in the issue body

- Customer email, name, or other PII.
- Tenant admin user_id (the tenant_slug is sufficient for engineering identification).
- Stack traces with internal infrastructure detail.
- Database row IDs that could be used to construct unauthorized queries.

The bug submission flow's PII redaction (per §32.7.6) applies to the issue body content.

Auto-generated by Self-Service Help v1. Submitted at ${submitted_at}.

32.7.5 Issue Creation Resilience

GitHub API failures must not lose the submission. If issue creation fails (network error, rate limit, GitHub down), the bug_submissions row stays in github_issue_state = 'pending' and a background job retries with exponential backoff. After 24 hours of failure, the row is marked 'failed', the user is notified via in-app banner, and a platform admin alert fires.

Once an issue is created, the issue_number and issue_url are written back to the bug_submissions / feature_requests row.

32.7.6 PII Redaction Before GitHub

Bug-report content can contain PII. Before any content is sent to GitHub:

- Run the v6 §22.4 redaction pipeline over all free-text fields.
- Zero-tolerance items (SSN, full credit-card numbers, passport numbers) — quarantine the submission, do NOT create the issue, alert a platform admin.
- Tolerable items (names, emails, phone numbers) — redact to [REDACTED-NAME] / [REDACTED-EMAIL] / [REDACTED-PHONE].
- Screenshots — strip EXIF metadata. Optionally run a vision-based PII check (Haiku vision pass) before upload. If detected, refuse upload and ask the user to redact and re-upload.

32.8 Confidence and Clarity Scoring

32.8.1 Why a Score

Not every bug report is actionable. A score lets the platform route high-quality reports into the auto-fix pipeline and lower-quality reports into the human-review queue without blocking either path.

The score is a single number 0.00–1.00 stored in bug_submissions.confidence_score. The factors that contributed are stored in bug_submissions.confidence_factors as JSONB for later analysis and threshold tuning.

32.8.2 Score Computation

The Help AI emits a structured assessment at the end of the bug flow. The score is the average of the following factor scores, each 0–1:

Factor	What it measures
specificity_of_location	How precisely ‘where in the platform’ identifies a route or feature
clarity_of_actual_behavior	Whether the actual behavior is described concretely
clarity_of_expected_behavior	Whether the expected behavior is described concretely
completeness_of_steps	Whether reproduction steps are complete and ordered
reproducibility_signal	Whether the user reports the bug as ‘always’ or ‘sometimes’ (always = higher)
environment_completeness	Whether browser/device info is captured

Weighting can be added later if a uniform average produces poor results. v1 uses uniform weighting.

32.8.3 Threshold Configuration

The auto-fix threshold is an environment variable BUG_AUTOFIX_CONFIDENCE_THRESHOLD with an initial value of 0.7. It can be changed without code deploy. Recommend recalibrating after the first 30 days of real submissions, once a distribution of real-world scores is observed.

32.8.4 Score Transparency

The submitting user sees their confidence/clarity score after submission, with a brief explanation of factors that scored low. This encourages quality submissions and lets the user revise before submission if they want.

Customers do not see the score — they see only ‘submitted’ confirmation with an issue link. This avoids putting non-technical users in a position of feeling judged by a number.

32.9 Bug Auto-Fix Pipeline

32.9.1 Overview

Bugs that score above the threshold and pass staging reproduction are eligible for an automated fix attempt by Claude Code. This pipeline is gated, observable, and never deploys to production without

human approval — the existing CI/CD §3 production approval gate stands.

32.9.2 Workflow

A new GitHub Actions workflow .github/workflows/bug-auto-fix.yml is added. Triggered on issue events:

- Issue opened with labels bug AND auto-fix-candidate.
- Workflow extracts the structured fields from the issue body (using a parser keyed on the section headers from §7.4).
- Workflow opens a fresh staging environment with the latest prod DB copy + fixups (per CI/CD §5).
- Workflow runs a reproduction attempt — initially a Playwright script generated by Claude Code from the reproduction steps; future versions may use a dedicated bug-repro framework.
- **Pre-fix repro gate:** the generated Playwright script is run against the CURRENT pre-fix code (on staging). The script MUST FAIL (i.e., demonstrate the bug). If the script passes against pre-fix code, the bug either doesn't reproduce or the script is faulty — in either case the auto-fix attempt is aborted. Add label `repro-script-faulty` and post a comment explaining the failure.
- If pre-fix repro fails as expected: add label `repro-confirmed`; invoke Claude Code via API to propose a fix; open a draft PR `fix/issue-{number} → dev` with the repro test included.
- **Post-fix repro gate:** the proposed fix is applied; the same Playwright script is run against the now-modified code. The script MUST PASS (i.e., demonstrate the bug is no longer reproducible). If the script still fails, the fix didn't fix the bug — abort PR creation, add label `fix-ineffective`, post a comment.
- If both gates pass: the draft PR is opened with the repro test included. PR review then verifies the fix is sensible (review is human; auto-fix only verifies the test pass/fail behavior is correct).
- If reproduction script generation fails entirely (Claude Code cannot produce a valid Playwright script from the issue text): add label `repro-failed`; post a comment summarizing what was tried; the issue stays open for human review.

32.9.2a Calls Worth Flagging

- **The two-gate pattern (pre-fix MUST fail, post-fix MUST pass) is the defense against the "auto-fixed but didn't actually fix anything" loop.** Without it, a faulty repro that passes against buggy code would let the auto-fixer "fix" nothing and the PR would CI-green its way through review.
- **The pre-fix gate runs against CURRENT staging code**, not against a fresh checkout. If the bug was already fixed by an earlier release, the gate catches that ("repro doesn't reproduce — bug may be already fixed"). The issue is closed as not-reproducible.
- **The post-fix gate runs against the proposed fix applied locally to the workflow runner.** Once the gate passes and the PR opens, normal CI runs against the full PR diff and may catch other regressions.

- **Both gates use the same Playwright script.** A fix that requires changing the test is suspicious.

32.9.3 PR Conventions

Auto-fix PRs are always:

- Draft state (not 'ready for review').
- Include the reproduction test in the same PR as the fix.
- Title format: 'fix: {first 80 chars of issue title} (#{issue_number})'.
- Body includes a link to the issue, a summary of the analysis, the proposed fix description, and a note that production deploy still requires human approval.
- Auto-fix PRs are flagged with a github label auto-fix-pr so the team can filter.

32.9.4 Manual Override

A platform admin can add the label needs-human-fix to any issue, which removes auto-fix-candidate and prevents the auto-fix workflow from triggering. Useful when the admin knows the bug is sensitive (security, compliance, customer-impact heavy) or when a previous auto-fix attempt produced a poor result.

32.9.5 Failure Modes and Their Handling

Failure mode	Handling
Staging DB copy fails	Skip repro; label repro-failed with explanation; alert platform admin
Playwright script invalid	Comment on issue with error; label repro-failed
Claude Code fix attempt times out	Close any partial PR; comment on issue; label repro-confirmed remains; needs human intervention
Fix PR fails CI	PR remains draft; engineer takes over
Fix PR review rejected	Engineer closes PR; issue stays open

32.9.6 Cost Controls

The auto-fix pipeline can consume meaningful Claude API budget. Controls:

- Per-day cap on auto-fix attempts across the platform — initial value 10/day. Beyond cap, additional candidates queue.
- Per-tenant cap — initial value 3/day. Prevents a single tenant from dominating the pipeline.

- Claude API key for the auto-fix workflow has its own spending limit configured in the Anthropic Console (separate from the production app key).
- All auto-fix Claude calls log to the AI cost-attribution system (v6 §27.12) under a synthetic ‘platform-autofix’ tenant identifier.

32.10 Customer Bug Flow

32.10.1 Trigger

The existing travel concierge persona gets a new intent recognizer for bug reports. Trigger phrases include (non-exhaustive):

- ‘this is broken’
- ‘something’s wrong with…’
- ‘i think there’s a bug’
- ‘this page is glitching’
- ‘the website crashed’

On detection, the travel concierge responds with: ‘It sounds like something might not be working right. Would you like me to file a bug report? I’ll ask you a few quick questions and our engineering team will take a look.’

32.10.2 Authentication Gate

Customer bug reporting requires the customer to be OAuth-authenticated (per v6 §17). An anonymous customer trying to file a bug is asked to sign in first. The original conversation context is preserved through the anonymous-to-authenticated transfer (v6 §11.6).

If the customer declines to sign in, the travel concierge offers an alternative: ‘Would you like me to summarize the issue and forward it to a human agent at {tenant business name}?’ This routes to the tenant’s existing escalation path (v6 §10.3 topic escalations) rather than to GitHub.

32.10.3 Flow Hand-off

If the customer accepts and is authenticated, the chat surface visually shifts (subtle banner: ‘Bug report mode’) and the bug flow takes over within the same conversation. The bug flow uses the same structured questions as §4.3, adapted in tone for end customers:

- ‘Where were you when you noticed this?’ (instead of ‘Where in the platform’)
- ‘What happened?’
- ‘What did you expect to happen?’
- ‘Can you walk me through exactly what you did?’
- ‘Did this happen once, or has it happened more than once?’

- ‘I’ll grab your browser info — does this look right?’
- ‘Want to send a screenshot? Drag it into the chat or upload.’

32.10.4 Issue Creation

The issue is created in the same GitHub repo as tenant-reported bugs (D-002 in MEMORY.md). Labels applied: bug, customer-reported. The submitter_user_id is the customer’s user_id (visible only to platform staff via the bug_submissions row, never exposed in the public GitHub issue body). The tenant_slug appears in the issue body so engineering can identify which tenant’s customer is affected.

32.10.5 Customer Confirmation

After submission, the customer sees: ‘Thanks — I’ve sent that to the engineering team. You’ll hear back if we need more info.’ The customer does not see the GitHub issue URL directly (it’s an internal artifact); they see a friendly reference ID.

32.10.6 Confidence Threshold for Customer Bugs

Customer-reported bugs follow the same confidence threshold for auto-fix eligibility. However, customer-reported bugs tend to have lower clarity scores than tenant-admin-reported bugs (different vocabulary, less technical specificity). The threshold value can be set independently for customer-reported bugs if real-world data shows the uniform threshold is poorly calibrated; v1 uses a single threshold for simplicity.

32.10.7 Resolution Notification

When the corresponding GitHub issue is closed, the customer optionally receives an in-app or email notification (depending on their preferences). The notification phrasing avoids technical detail: ‘The issue you reported has been resolved. Thanks for letting us know.’

32.11 Abuse Monitoring

32.11.1 New Dimension

v6 §27 defines an abuse-monitoring framework with multiple dimensions (AI cost, chat volume, email volume, group invitations, RAG submissions). This feature adds a new dimension: help_submission_rate.

32.11.2 Initial Thresholds

Per-tenant per-day submission counts (bugs + feature requests combined). These are initial values; recalibrate after first 90 days of usage:

State	Threshold (submissions/day)	Action
ok	≤ 19	No action
		In-app banner to

soft1	20–49	platform admin; no tenant notification
soft2	50–99	Email to tenant owner; throttle to 1 submission per 10 minutes for that tenant
hard	≥ 100	Block further submissions for the rest of the day; alert platform admin

32.11.3 Cost Attribution

Help AI calls attribute to the tenant whose user is interacting, exactly as customer-facing AI calls do (v6 §27.12). This means a tenant with many Help AI sessions will see those costs in their tier consumption.

32.11.4 Customer-Bug-Specific Rate Limit

In addition to per-tenant rate limits, individual customers face a per-customer limit on bug submissions: 5 per day per customer. This blocks a single customer from spamming GitHub via the AI. Exceeding the limit triggers a polite refusal: ‘You’ve reported a few issues today already — our team will get to them. Try again tomorrow if you find something else.’

32.12 Permissions

32.12.1 Role Mapping

Per v6 §26, the role model is:

Role	Can view docs	Can use help flow	Can submit bug	Can submit feature
tenant_owner	Yes	Yes	Yes	Yes
tenant_agent	Yes	Yes	Yes	Yes
tenant_billing_admin	Yes	Yes	Yes	Yes
customer (authenticated)	No	No	Yes (via concierge)	No
customer (anonymous)	No	No	No	No
platform_super_admin	Yes (all tenants)	Yes	Yes (tagged platform-internal)	Yes
platform_support	Yes (all tenants)	Yes (read-only view)	Yes (on behalf)	Yes
platform_compliance	Yes	Yes	Yes	Yes
platform_finance	Yes	No	Yes	Yes

32.12.2 Customer Feature Requests

Customers cannot directly submit feature requests via this v1 release. Their feedback can still reach the platform via the tenant's existing escalation path, where a tenant agent may translate it into a feature request through the tenant admin flow. This is a deliberate scope choice; customer-direct feature requests can be added in a future version once the volume model for customer bugs is understood.

32.13 Privacy and Security

32.13.1 PII Handling

Every free-text field captured by the bug or feature flow passes through the v6 §22.4 PII redaction pipeline before:

- Being stored in bug_submissions or feature_requests (redacted form is stored; raw form is discarded).
- Being sent to GitHub as issue body.
- Being indexed or retained beyond the redaction step.

32.13.2 Screenshots

Uploaded screenshots are subject to:

- EXIF metadata stripped on upload.
- Stored in tenant-scoped Supabase Storage with signed-URL access only.
- Optional vision-based PII detection before issue attachment — initial behavior: warn the user, do not block. After 90 days of data, evaluate moving to block-on-detection.

32.13.3 Audit Logging

Audited events:

- Help session opened, closed, escalated.
- Bug or feature submitted (rows are themselves audit records).
- GitHub issue creation (success and failure).
- Auto-fix workflow triggered, succeeded, failed.
- Platform admin override (needs-human-fix label applied).
- Feature request decision (accepted/rejected/deferred/duplicate).

Retention follows v6 audit-log retention: 7 years.

32.13.4 Tenant Isolation

Help sessions, bug submissions, and feature requests are tenant-scoped via RLS. A tenant admin can see only their own tenant's submissions. Platform admins can see across tenants. GitHub issues are not tenant-scoped (they live in a single repo) but the tenant

identifier is hashed in any externally-visible portion of the issue body, with the plaintext tenant ID stored only in the platform's bug_submissions row.

32.14 Environment Variables

New variables added by this feature. Follows the v6 §28 conventions.

Variable	Required	Secret	D
GITHUB_APP_ID	Yes	No	G id
GITHUB_APP_PRIVATE_KEY	Yes	Yes	G p (P
GITHUB_APP_INSTALLATION_ID	Yes	No	G in id (p in
GITHUB_REPO_OWNER	Yes	No	O th re
GITHUB_REPO_NAME	Yes	No	N is
BUG_AUTOFIX_CONFIDENCE_THRESHOLD	Yes	No	S th fc el (d
BUG_AUTOFIX_DAILY_PLATFORM_CAP	Yes	No	P w ca fi (d
BUG_AUTOFIX_DAILY_TENANT_CAP	Yes	No	P da au at (d
CLAUDE_CODE_API_KEY	Yes	Yes	A au C in se fr ap
HELP_DOCS_CACHE_TTL_SECONDS	No	No	T ca P ex (d 3(

32.15 Phased Rollout

32.15.1 Phase Alignment

v6 §31 defines four phases (0–3). This feature lands in phases as follows:

Phase	What ships
Phase 1	Tenant admin help flows (help / bug / feature). Documentation viewer with onscreen + print + PDF download. GitHub integration. No auto-fix yet — manual triage for all submissions.
Phase 2	Customer bug flow via the travel concierge persona. .docx download. Auto-fix pipeline enabled. Abuse monitoring dimension turned on with enforcement.
Phase 3	Threshold recalibration based on real submission data. Optional customer feature requests if data shows volume warrants. Vision-based screenshot PII detection moves from warn to block if calibration supports it.

32.15.2 Phase 1 Done Definition

- At least 5 doc sections written and approved.
- All three help flows tested end-to-end on staging.
- At least one real bug filed via the flow and visible as a GitHub issue.
- PDF download produces a readable, branded document.
- PII redaction verified by submitting a test bug containing fake SSN.

32.15.3 Phase 2 Done Definition

- Customer bug flow tested with a real authenticated test customer.
- Auto-fix workflow successfully reproduces a synthetic bug on staging.
- Auto-fix workflow opens a draft PR with proposed fix; engineer review confirms the fix is reasonable.
- Abuse monitoring dimension active and enforcing for non-platform tenants.

32.16 Calls Worth Flagging

Design trade-offs in this section that may need revisiting:

- **GitHub as the issue system.**** **Locks bug/feature handling to GitHub. If the engineering team ever migrates to Linear, Jira, or another tracker, the issue-creation layer (§7) has to be re-implemented. The repository abstraction is concentrated in apps/main/src/lib/github/, so the swap is contained — but it is a swap, not a config toggle.
- **Uniform confidence threshold for customer vs tenant-admin bugs.**** **Customers and tenant admins have different bug-reporting fluency. A single threshold may be miscalibrated for one side. Phase 3 may want a per-source threshold.
- **Auto-fix Playwright reproduction.**** **Bugs that don't involve UI interaction (e.g., a background-job failure, an API contract mismatch) won't reproduce in a UI-driven script. The reproduction layer needs to support multiple reproduction types over time. v1 supports UI-driven repro only.
- **Customer-reported feature requests are not in v1.**** **May create frustration if customers see they can report bugs but not request features. Worth measuring tenant feedback after Phase 2.
- **Help docs as part of the release pipeline.**** **Means a doc typo correction requires a release. Acceptable while the team is small; revisit if docs become a frequent edit surface.
- **Screenshot PII detection starts in warn-only mode.**** **If a customer attaches an image with PII visible (e.g., a passport photo by accident), the platform allows the upload with a warning. A later phase may flip this to block-by-default.
- **Help_doc_versions cache invalidation on every release.**** **Cache key includes the code version. Every deploy invalidates the cache. Acceptable while deploys are weekly or less frequent; if deploys become daily, cached generation cost becomes nontrivial.
- **No SLA on auto-fix turnaround.**** **An auto-fix PR may sit in draft for a long time before a human picks it up. The PR queue can grow. v6 doesn't have an SLA framework; if one is added later, auto-fix PRs should be included.

Appendix A — Fresh-Build Notes

This appendix is for the engineer who picks up the codebase six months from now and asks 'why was this done this way?' or who's tasked with building it from scratch. It captures the choices, conventions, and traps that aren't obvious from the per-section spec text but matter for getting the implementation right.

A.1 Things That Look Like Bugs But Aren't

A short list of behaviors that will look wrong if you don't know they're deliberate:

- Monotonic abuse-monitoring state within a calendar month. A tenant whose usage drops back below threshold remains in the elevated state until next month. This is intentional anti-oscillation.

See §27.7.

- No down-migrations. The schema is forward-only. Rollback is via code rollback (Vercel) or compensating forward migration. See §29.6.
- The platform structure is strictly two-level (platform and tenant). Sub-hosts who engage subcontractors track that internally; the platform has no relationship with subcontractors and does not facilitate payments to them. See §1.5 and §14.3a.
- Tenant-promoted RAG chunks don't move; they're duplicated. Promotion to global creates a new global chunk. The original tenant chunk remains. This preserves tenant continuity and audit. See §22.9.
- DOBs are stored, ages are computed. Ages drift; DOBs don't. Anywhere you see age math, look for the DOB upstream. See §11.5.
- The same OAuth user has multiple public.users rows across tenants. Don't dedupe across tenants. The user has separate memory, separate CRM identity, separate everything in each tenant they've engaged with. See §3.7.
- Pricing chunks auto-cap at authority 0.55. Even when from an official source. This is deliberate against price staleness. Override exists but logs reason. See §6.8.
- The fallback email adapter returns synthetic booking refs. Strings like EMAIL-{tenant_id}-{ts}. These will look weird in admin views until you know they mean 'not yet reconciled with real host system.' See §13.6.
- Memory extraction happens every 10 turns, not every turn. Don't optimize for per-turn extraction; that's not the design. See §11.2.

A.2 Multi-Tenancy Discipline

This is the single most error-prone area of the codebase. Cross-tenant bugs are catastrophic. A few rules:

- Every tenant-scoped table has tenant_id. No exceptions. If you find a tenant-scoped concept that doesn't have it, that's a schema bug, not a 'we'll figure it out.'
- Every API handler runs assertPermission() at entry. No exceptions. Don't add 'internal endpoints' without auth — those are how cross-tenant bugs ship.
- RLS is defense in depth, not the only defense. Application code also filters by tenant_id. Both layers must be correct.
- Service role bypasses RLS. Use it sparingly (webhooks, scheduled jobs, admin operations). When used, the code must filter explicitly. Audit-log every service-role mutation.
- Test cross-tenant access on every new route. Add a Playwright probe. Cross-tenant probe tests run in CI per §30.8.
- Tenant resolution happens once per request, in middleware. Don't re-resolve mid-request.

- ‘Anonymous’ is a user-state, not a tenant-state. Anonymous chat still resolves to a tenant (by host). Don’t conflate the two.

A.3 Authorization vs. Authentication

The codebase treats these as distinct concepts:

- **Authentication** ** **= ‘who is this person?’ Handled by Supabase Auth. Returns a verified auth_user_id.
- **Authorization** ** **= ‘is this person allowed to do this?’ Handled by assertPermission() against rules defined in code.

Routes never check ‘is the user signed in?’ without also checking ‘is the user authorized for this action?’ The two questions get answered together. A route that only checks authentication will eventually leak.

A.4 The Three Date/Time Conventions

- All timestamps stored as TIMESTAMPTZ in UTC.
- All dates without time stored as DATE (no time zone). For example: sailing date is a DATE — the ship sails on June 15 regardless of where the customer is.
- Tenant-displayed times convert to tenants.timezone. Customer-displayed times convert to the customer’s resolved time zone. Never assume UTC at the display layer.

Implication: never store ‘5pm’ as a string. Either it’s a TIMESTAMPTZ with an explicit instant, or it’s a DATE + a tenant-relative time interpreted at display.

A.5 IDs Everywhere

- All primary keys are UUIDs (gen_random_uuid()). No sequential IDs. This prevents enumeration attacks and lets the platform reseat / merge data without ID collision.
- Foreign keys cascade only where data ownership is obvious (e.g., users to tenant cascades; bookings to users does NOT cascade — deleting a user does not delete their bookings).
- External-system IDs (Stripe IDs, host booking refs, etc.) live alongside the UUID, never replace it. The UUID is platform-internal truth.

A.6 Encryption Boundaries

Three layers of encryption exist; they protect against different threats:

- In transit (TLS). Standard. Vercel handles this.
- At rest (Supabase-managed). Standard. Supabase handles full-disk encryption.

- Application-layer encryption (you). This is the one to think about. Used for credentials (host adapter API keys, Resend API keys, anything that grants real-world authority). Uses AES-256-GCM with a per-record IV. Key lives in env var. See §13.5 and §26.6.

The reason for layer 3: a DB-level breach (someone exfiltrates the DB) reveals everything in layers 1-2 in plaintext. Layer 3 keeps credentials encrypted even if the DB is dumped.

A.7 Idempotency Is Not Optional

Critical mutations accept an Idempotency-Key header. Stripe webhooks, booking submissions, commission writes — all of these must be idempotent. Cached for 24 hours.

The reason: networks fail. Clients retry. Stripe retries. Inngest retries. Without idempotency, a single conceptual operation becomes two real ones. Two commissions credited. Two bookings submitted. Two emails sent.

When in doubt: make it idempotent. The cost is small (24-hour cache row). The cost of getting it wrong is real money.

A.8 Why Two Supabase Projects

The RAG service has its own Supabase project. This is not ‘scale-driven’ — it’s a design choice:

- RAG schema can evolve without main app migrations
- RAG service can be redeployed without main app downtime
- Security boundary is explicit (RAG service has its own service-role key)
- Vector query load patterns are different from transactional load patterns
- Audit and ownership are cleaner

Cost: two Supabase project bills, slightly more operational overhead. The tradeoff is intentional. Don’t merge them in the name of simplicity.

A.9 Why JWT for Inter-Service Auth (Not Shared Secret)

The main app calls the RAG service many times per chat turn. Options considered:

- Shared secret in header. Simple, but every endpoint has to verify the same thing. Rotation is hard. No per-call context.
- Mutual TLS. Heavy. Vercel function cold starts make it worse.
- JWT signed by main app, verified by RAG. Carries per-call context (tenant, user, persona). Replay-protected via jti. Rotation manageable with overlap windows.

JWT wins. See §8.3.

A.10 The Schema-First Mindset

Schema is the source of truth. Type definitions in code mirror it (generated where possible). When schema disagrees with code, schema wins; fix the code.

A corollary: schema reviews matter. Adding a column without thinking about the type, nullability, indexes, RLS, and migration backward-compatibility is how broken schema ships. Treat schema PRs with the same care as security PRs.

A.11 Why React Email for Templates

Three reasons:

- Type-safe templates. Tenant branding context flows through as typed props. No string interpolation bugs.
- Components compose. The BrandedLayout component embeds all the tenant-branding boilerplate. Templates focus on content.
- Plain-text alternates generate automatically. Email clients that don't render HTML get a usable fallback.

The alternatives (raw HTML templates, MJML, etc.) all sacrifice one of these. React Email is the local maximum.

A.12 Tone Matching Is a Feature, Not a Hack

The 5-level tone scale (§24.5) is part of the persona design, not an afterthought. The supervisor's tone_drift check exists to catch deviations — too casual on serious topics, too formal where rapport has been built. The customer's rapport_tone_level in memory persists across conversations.

Engineers shouldn't strip this out as 'complexity.' It's a deliberate part of why the AI doesn't feel robotic. If a tenant's customers complain about tone, look at level configuration first, not 'remove tone matching.'

A.13 Anti-Patterns to Avoid

A non-exhaustive list of things that look reasonable but cause problems:

- Caching tenant data in module-level globals. Tenant context changes per request. Module globals leak.
- Building 'internal admin shortcuts' that bypass assertPermission. Convenience for the operator becomes a vulnerability when forgotten.
- Logging full prompts including tenant content. PII leak. Use IDs.

- Holding a DB connection across an AI streaming response. Long-lived connections starve the pool. Stream from buffer, not from open connection.
- Trusting host adapter responses without sanity-checking. The adapter returns what the host sent. The host can send garbage. Validate.
- Letting ai_mode = 'disabled' mean 'no AI calls ever.' It still means 'no customer-facing AI.' Background AI (memory extraction, supervisor preflight where applicable) still runs.
- Putting Stripe price IDs in code. They're environment-specific (test mode vs live mode have different IDs). Env vars per §28.7.
- Assuming the supervisor catches everything. It catches most. The remainder gets surfaced via thumbs-down, eval suite, and customer feedback. Defense in depth.
- Treating the fallback email adapter as a 'real' adapter for testing. It's a degraded mode, not a test mode. Use proper test mode and adapter mocks.

A.14 Naming Conventions

- Tables: snake_case, plural (bookings, not booking)
- Columns: snake_case, singular (booking_id, not bookings_id)
- Foreign keys: {referenced_table_singular}_id (tenant_id, user_id)
- JSON fields: snake_case keys for consistency with DB columns
- TypeScript: camelCase variables, PascalCase types/classes
- Routes: kebab-case segments (/api/group-bookings/…)
- Env vars: SCREAMING_SNAKE_CASE
- React components: PascalCase
- Feature flags: SCREAMING_SNAKE_CASE matching env var convention

Type generation handles the snake_case ↔ camelCase boundary at the DB layer. Don't fight it.

A.15 The 'Don't Decide Now' Items

Some choices were deliberately deferred to be made in implementation context. Don't backfill them into the spec — they need real-world signal to decide right:

- Image generation provider (Replicate SDXL vs OpenAI DALL-E 3): depends on cost-per-image and quality at the time of implementation. See §2.3.
- Native flow vs host widget for booking (§20): depends on host agency selection.
- Inngest tier: depends on job volume.

- Anthropic plan tier: depends on launch chat volume.
- Whether to add Redis cache: depends on observed query patterns. None at launch; add if needed.
- Read replicas for Supabase: depends on observed load. None at launch.

Building the spec to dictate these would have made it wrong on day one of operation. They're decisions for the engineer with their hands on production metrics.

A.16 Recommended Reading Order (For New Engineers)

If someone needs to learn the system, read in this order:

- §1 (overview), §2 (stack), §3 (multi-tenancy) — get the model
- §5 (main schema), §6 (RAG schema) — get the data
- §9 (personas), §10 (supervisor), §21 (RAG consumer side), §21.10 (hallucination defense) — get the AI behavior
- §13 (host abstraction), §14 (commissions) — get the money path
- §17 (auth), §25 (privacy), §26 (security) — get the safety
- §27 (abuse monitoring) — get the operational constraints
- Everything else as relevant to current task

The temptation is to read in section order. Don't. The dependency order above gets you productive faster.

A.17 When the Spec Is Wrong

The spec will be wrong sometimes. Reality always surprises specs. When this happens:

- If a section's stated behavior is contradicted by real observations, update the spec. Don't keep the code matching a wrong spec.
- If a 'Call Worth Flagging' item turns out to be the bigger problem than the main design, promote it. Move the flag to the main text and explain.
- If a 'Deferred' item becomes urgent, document the reactivation reason in MEMORY.
- If MEMORY contains a decision that the spec contradicts, MEMORY wins (it's more recent). Update the spec to match.

The spec is a living document. Out-of-date specs are worse than no specs.

Appendix B — Glossary

Quick-reference definitions for terminology used throughout the spec. Entries grouped by domain rather than alphabetized — closely-related terms appear together for context.

B.1 Platform Structure

Platform — The system as a whole, operated by the platform owner. Hosts multiple tenants. Also the legal entity that has the relationship with the host agency, Stripe, Anthropic, and other vendors.

Tenant — A direct account on the platform. One of three types: BYO-host, sub-host, or the platform's own automated agency. No tenant nests beneath another — the platform's structure is strictly two-level.

BYO-host tenant — An independent travel agent who has their own host agency relationship and subscribes to the platform as SaaS. They bring their own host credentials; commissions flow through their existing arrangements without platform intermediation.

Sub-host tenant — An independent travel agent who doesn't have their own host agency relationship and operates under the platform's host agency. The platform takes a commission split and remits the remainder via Stripe Connect Express.

Subcontractor — A person a sub-host engages to help operate their agency. NOT a tenant on the platform. The platform has no relationship with subcontractors — they're tracked privately inside the sub-host's tenant for internal bookkeeping only.

Platform tenant — A special tenant: the platform operator's own automated travel agency (Prong 1). Operates like any other tenant but is operated by the platform owner directly.

Three prongs — The three business models the platform supports: (1) the platform's own automated agency, (2) BYO-host SaaS, (3) sub-host (white-label).

B.2 People & Roles

Customer (end traveler) — The person being served by a tenant — the one who books a cruise. Has an account on the platform; may interact with multiple tenants over time, each with separate identity and memory.

Tenant owner — The operator of a tenant. Has full admin access to that tenant. Different from the platform owner.

Tenant agent — A sub-user of a tenant who assists with customer service. Permissions narrower than tenant owner. Multiple per tenant on BYO Agency tier.

Platform super admin — Top-level platform operator with full access to admin console.

Platform compliance — Role responsible for reviewing tenant onboarding submissions.

Platform finance — Role responsible for reconciling commissions and payouts.

AI persona — One of the six AI travel advisors a customer can interact with: Marcus, Marco, Priya, Dave, Maya, Jenny.

Group coordinator — A customer who is organizing a group booking — created the group, manages invitations.

Group invitee — A customer invited to a group booking by a coordinator.

B.3 Money

Commission — The amount paid by the host agency for a booking. Computed as commission rate × commissionable fare.

Commissionable fare — The portion of a booking's total cost that earns commission. Distinct from total. Excludes taxes, port fees, insurance, and some addons.

Host booking fee — A per-booking fee that host agencies often deduct from gross commission before the platform's share is calculated. May be flat, percent, or tiered.

Net commission — What the platform actually receives after host booking fees are deducted. The amount the two-party split is applied to.

Two-party split — The commission allocation: platform retains its tier-defined percentage of net commission; sub-host receives the remainder. There is no hierarchy or chain walk.

Tier-defined percentage — The percentage of net commission the platform retains, set by the sub-host's tier (Starter 30%, Pro 20%, Agency 10%).

Hold period — The time between commission receipt and payout availability for a sub-host. 7 / 3 / 0 days by tier (Starter / Pro / Agency). Cushions against clawback risk.

Pending balance — Amount accrued but still in hold period. Not yet payable.

Available balance — Amount past hold period; eligible for the next scheduled payout.

In-transit balance — Amount currently in a Stripe transfer; not yet settled to the bank.

Payout — The Stripe Connect transfer that moves money from the platform's account to a sub-host's connected bank.

Clawback — Reversal of a previously-paid commission, typically due to customer cancellation. May happen during hold period (deducted from pending) or after payout (Stripe reversal or contractual recovery).

1099-NEC — US tax form Stripe Connect Express files automatically for sub-hosts earning $600+/year. Platform's job is to ensure all payouts flow through Connect.

Stripe Connect Express — Stripe's marketplace product for handling sub-host KYC, banking, and payouts. Express level: simpler onboarding, Stripe owns more of the dashboard.

Subscription — The platform's SaaS fee charged via Stripe Subscriptions, separate from commission flow. BYO-host pays only this; sub-host pays this AND has commission split.

Trial period — 30 days, starting at tenant activation (not signup), during which no subscription charge applies.

B.4 AI Behavior

Persona — An AI character with defined expertise, voice, and visual identity. Six total: Marcus, Marco, Priya, Dave, Maya, Jenny.

Persona overrides — Per-tenant customization of personas: display name renaming (Pro+ tier), system prompt addendum (Agency tier only), disabling.

System prompt — The instruction text given to Claude that defines a persona's behavior. Built from three layers: base persona block, platform constraints, optional tenant addendum.

Prompt addendum — Tenant-supplied additional persona instructions. Agency tier only. Haiku-screened for safety before save.

Supervisor — The safety layer that runs around every AI response: hallucination check, persona drift, promise detection, arithmetic check, compliance keyword, tone drift, topic escalation.

Preflight check — A supervisor check that runs before an AI response is surfaced to the customer. Pass = release; flag = regenerate or escalate.

Tone drift — A supervisor check that catches tone-topic mismatches (e.g., casual on serious topics) or tone changes inconsistent with configuration.

Tone level — One of five levels customers can experience: formal, professional, casual, friendly, matched-energy. Per-tenant cap (default 3) and per-customer memory of current level.

Topic-level escalation — Escalating one topic in a customer's conversation to human while AI continues to handle unrelated topics. Different from conversation-level escalation.

Kill switch — Platform-wide ability to pause AI responses (per §10.6). Customer-facing chat falls back to 'human will be in touch.'

Hallucination defense — The 8-layer stack of mechanisms that reduce AI factual errors: persona instructions, RAG grounding, hallucination check, confidence floor, promise detection, arithmetic check, customer feedback, escalation safety net.

Tool use / tools — Functions the AI can call: search_host_inventory, get_customer_context, generate_quote, collect_booking_details, escalate_to_human, update_memory.

AI mode — Tenant-level setting: autonomous (AI replies directly), draft_only (AI drafts, tenant agent reviews and sends), disabled (no AI).

Disclosure (AI) — Required notice that the customer is interacting with AI. Persistent UI banner; persona discloses on direct question.

B.5 RAG (Retrieval-Augmented Generation)

RAG — The system that retrieves relevant knowledge chunks and includes them in AI prompts to ground responses in source material.

Knowledge chunk — A unit of text (~500 tokens) with metadata, embedded as a vector, stored in the RAG service's database.

Embedding — A 1536-dimension vector that captures semantic meaning of a chunk. Used for similarity search.

Scope — Visibility of a chunk: global (all tenants) or tenant (only the owning tenant). The platform's structure is strictly two-level — no inheritance, no ancestor walks.

Authority — A 0-1 score indicating how trustworthy a chunk is. Five tiers: low (0.30-0.54), tenant_default (0.55-0.69), tenant_verified (0.70-0.84), official (0.85-0.95), authoritative_override (0.96-1.0).

Recency — A computed score reflecting how fresh a chunk is. Decays over time; category-specific halflives.

Composite confidence — Weighted combination of match score, authority, and recency. Used for filtering at retrieval.

Confidence floor — Minimum composite confidence (0.35) for a chunk to be included in a prompt. Filters out weak matches.

Promo lifecycle — States a promotional chunk moves through based on dates: not_started → open_to_new → closed_to_new → expired. Sell-by and sail-by dates drive transitions.

Pricing authority cap — Pricing chunks auto-cap at 0.55 authority regardless of source. Manual override available with logged reason.

Promotion (RAG) — Action where platform admin moves a tenant chunk to global scope. Permanently raises the source tenant's RAG cap by 25.

Tenant cap (RAG) — Per-tenant total cap on tenant-specific chunks (relevance < 0.6). Base value by tier; each global promotion adds 25 permanently.

Queue-eligible item — A submission with relevance ≥ 0.6, awaiting platform admin global-review decision. Doesn't count against tenant cap while pending.

Authority feedback loop — Customer thumbs-up/down feeds small (bounded ±0.05) adjustments to chunk authority over time.

B.6 Multi-Tenancy & Security

Tenant resolver — Middleware that maps a request's hostname to a tenant context (subdomain, custom domain, or platform-direct).

RLS (Row-Level Security) — PostgreSQL feature that enforces tenant scoping at the database level. Defense in depth — application code also filters.

auth_user_in_tenant(tenant_id) — RLS helper function used in policies to check whether the authenticated user has a public.users row in the target tenant.

assertPermission() — Function called at every API handler entry that verifies the user is authorized for the action. Logs to audit_log.

Service role — Supabase service-level credential that bypasses RLS. Used by webhooks and scheduled jobs. Strict discipline — all use audit-logged.

Cross-tenant access — Attempted access from tenant A to tenant B's data. Should always fail. Tested extensively.

Audit log — Append-only table recording every significant action: actor, action, target, outcome. 7-year retention. Encrypted at rest.

Anonymization — Replacing PII with placeholder values while preserving required-retention records. Triggered on customer deletion requests for booking/commission/audit records that cannot be fully deleted.

Application-layer encryption — AES-256-GCM encryption of credentials (host adapter API keys, tenant Resend keys) in the database. Per-record IV. KMS key in env var.

JWT (inter-service) — JSON Web Token signed by main app, verified by RAG service. RS256, 5-minute TTL, jti-based replay protection.

B.7 Booking & Group

Quote — A trip price summary sent to a customer. Has lifecycle states: draft, sent, viewed, accepted, declined, expired, converted. Becomes a booking on acceptance.

Booking — A confirmed-to-be-purchased trip. Has lifecycle states: draft, submitted, pending_host_review, confirmed, modified, cancelled, sailed, completed.

Host adapter — The pluggable layer that translates platform booking requests to a specific host agency's API. Each adapter implements the HostAgencyClient interface.

Fallback email adapter — A degraded-mode adapter that emails structured booking details to a configured destination. Used when no real adapter is configured or available. Returns synthetic EMAIL-{tenant}-{ts} booking refs.

Provider booking ref — The reference number assigned by the host agency / cruise line for a confirmed booking. Stored alongside platform's UUID.

Sailing date — The DATE the ship departs. Stored as DATE (no time zone) — the ship sails on this calendar date regardless of where customer is.

Group booking — A coordinated booking involving multiple cabins, organized by a group coordinator. Has lifecycle states: planning, active, closed, sailed, cancelled.

Group invitee — A customer invited to a group. May or may not be a platform user yet at the time of invitation.

Invitation token — HMAC-signed token in invitation URL that authenticates the invitee landing page without requiring login. Expires sailing_date + 30 days.

RSVP — Invitee's response state: pending, interested, not_going, booked.

Coordinator portal — The interface where group coordinators manage their group: invitees, edits, hero image, preview, forum moderation.

Forum (group chat) — Threaded asynchronous discussion area per group. AI-moderated. Text only at launch.

B.8 Operational

Inngest — Background job platform. Handles scheduled jobs (daily resets, recompute), event-driven flows (booking submitted → commission tracking), and retry queues.

Resend — Email sending provider. Two patterns: tenant's own account (Pattern A) or platform's account with tenant CNAME (Pattern B, default).

SSE (Server-Sent Events) — Streaming protocol used for chat responses. Client subscribes; server streams response tokens. Reconnect via Last-Event-ID.

Sandbox mode — Tenant testing mode where bookings go through the fallback adapter, no real commission records are created, and Stripe is in test mode. Switching to live requires explicit confirmation.

Onboarding stage — Where a tenant is in the signup → review → active flow. Persisted on tenants row.

Override (usage) — Admin-granted exception to a tenant's normal usage threshold (per §27.9). Time-bounded, reason-required, audit-logged.

Boot-time env verification — Service startup check that fails-loud if required env vars are missing or invalid. Per §28.19.

B.9 Compliance & Legal

ICA (Independent Contractor Agreement) — The substantive contract between platform and sub-host. Distinct from ToU. Accepted with electronic signature (typed legal name).

ToU (Terms of Use) — Platform-level legal document. Every user accepts on signup.

Privacy Policy — Platform-level legal document. Every user accepts on signup.

AI Liability Disclaimer — Platform-level legal document. Every user accepts on signup. Acknowledges AI may make mistakes, AI is not a licensed travel agent, etc.

CAN-SPAM — US email marketing law. Requires physical address, accurate from-line, unsubscribe link, prompt unsubscribe processing.

CCPA — California Consumer Privacy Act. Defines customer rights: know, delete, correct, opt-out of sale, non-discrimination.

Seller of Travel (SoT) — State-level registration required to sell travel in some states (CA, FL, WA, etc.). Handled via ICA clause referring to platform's host agency registration. Deferred per-tenant verification pending host selection.

E&O Insurance (Errors & Omissions)** — Professional liability insurance. Handled via ICA clause referring to platform's host agency coverage where applicable. Deferred pending host selection.

Versioned consent — System where every legal document has a version; consents are recorded per (user, document, version, action). Re-prompt on version change.

Suppression (email) — A recipient × tenant pairing flagged to skip email sends. Types: hard_bounce, complaint, unsubscribe_all, unsubscribe_marketing, unsubscribe_news, manual_suppress.

B.10 Data Model Concepts

UUID — Primary key format used everywhere. Prevents enumeration; allows merge/reseat without collision.

Tenant-scoped table — A table whose rows belong to a specific tenant. Has a tenant_id column with RLS policy enforcing isolation.

DOB-not-ages — Storage convention: dates of birth (DATE) are stored, never ages. Ages computed at sailing date for booking validation. Estimated DOBs flagged with date_of_birth_is_estimated.

Memory — Per-customer, per-tenant structured context used to prime AI conversations: preferences, family composition, accessibility needs, dietary restrictions, etc. Customer-controlled (view, edit, delete, opt-out).

Conversation — A continuous chat session between a customer and an AI persona. Has summary, status, message count.

Message — A single turn in a conversation: user message or assistant response. Carries persona, RAG chunks used, supervisor findings, feedback score.

Contact (CRM) — A structured record of a person known to the tenant. May or may not have a corresponding user account. Multi-traveler bookings reference contacts.

Contact relationship — Directional link between two contacts: spouse, partner, parent_of, child_of, sibling, friend, colleague. AI inferred or manually added.

B.11 Phases & Status

Phase 0 — 'Foundation' — infrastructure setup. Per §31.3.

Phase 1 — ‘First Revenue’ — Prong 1 alive with real customers, real commissions. Per §31.4.

Phase 2 — ‘Multi-Tenant’ — Prongs 2 and 3 open; full tier matrix. Per §31.5.

Phase 3 — ‘Scale’ — deferred items shipped; performance work. Per §31.6.

Sailed — A booking lifecycle state. Group bookings become read-only on the travel start date.

Suspended (tenant) — Tenant state: read-only, no new bookings; existing commissions continue to settle.

Terminated (tenant) — Tenant state: closed; data anonymized per retention policy.

Active (tenant) — Tenant state: fully operational; can take customer interactions and bookings.

B.12 Self-Service Help Terms

Terms specific to the Self-Service Help feature (§32). See §32 for the full feature specification.

Help AI — The AI persona that runs the help / bug / feature chat flows defined in §32. Distinct from the travel concierge personas.

Help flow — One of three modes the Help AI runs in: ‘help’ (open Q&A), ‘bug’ (structured bug-info gathering), ‘feature’ (structured feature-request gathering).

Help session — A single user session in any help flow. One row in the help_sessions table.

Auto-fix — A pipeline that attempts to reproduce a bug on staging and propose a fix via Claude Code, gated by a confidence/clarity threshold.

Confidence/clarity score — A 0–1 score computed by the Help AI at the end of the bug flow, capturing how actionable the bug report is.

Customer-reported bug — A bug filed by an end customer through the travel concierge persona. Labeled ‘customer-reported’ on the corresponding GitHub issue.

Tenant-admin-reported bug — A bug filed by a tenant admin or agent through the admin console help flow. Labeled ‘tenant-admin-reported’.

Repro-confirmed — Label applied when the auto-fix pipeline reproduced the bug on staging.

Repro-failed — Label applied when the auto-fix pipeline could not reproduce the bug; awaiting human review.

---

1. a-z0-9↩